

BUỔI 5 · LÝ THUYẾT

Thiết kế kiến trúc hệ thống và Mô hình dữ liệu

Chuyên đề tốt nghiệp 1

Giảng viên chính HK261: **TS. Nguyễn Trí Hải / ThS. Nguyễn Minh Tân / ThS. Trần Công Thanh**

Học kỳ 1, năm học 2026 – 2027

Từ nguyên lý kiến trúc đến mô hình dữ liệu

01

Nguyên lý kiến trúc và tách trách nhiệm

Kiến trúc là gì và không phải là gì; kiến trúc phân lớp; ba mẫu thường dùng (Layered, MVC, Repository); phụ thuộc một chiều.

02

Tiêu chí thiết kế tốt và cách đọc một sơ đồ

Năm tiêu chí đánh giá; cách lập luận lựa chọn kiến trúc gắn với yêu cầu phi chức năng; đối sánh hai phương án cho cùng một yêu cầu.

03

Nguyên tắc mô hình hóa dữ liệu

Thực thể – thuộc tính – quan hệ; khóa chính và khóa ngoại; chuẩn hóa và khi nào chủ ý phi chuẩn hóa; bốn lỗi mô hình dữ liệu phổ biến.

04

Wireframe và mockup

Vai trò của wireframe; mức độ chi tiết cần thiết cho BT1; ba màn hình phải có; nhất quán giữa wireframe và đặc tả.

05

Tài liệu thiết kế (SDD) và mức rút gọn của học phần

Cấu trúc một bản SDD đầy đủ; phần nào áp dụng ở Chuyên đề tốt nghiệp 1; phần nào để lại cho Chuyên đề tốt nghiệp 2.

06

Phần chuyên ngành và chốt yêu cầu BT1

ERD chuẩn 3NF + SQL DDL [SE] · lược đồ star/snowflake và SCD [DA] · ML pipeline và dataset schema [AI]. Nhắc lại toàn bộ 5 thành phần BT1.

Kiến trúc là tập hợp các quyết định khó đảo ngược nhất về hệ thống.

- Kiến trúc KHÔNG phải là bản vẽ đẹp, cũng không phải là danh sách công nghệ. Nói "hệ thống em dùng React, Node và PostgreSQL" là kể tên công cụ, chưa phải là mô tả kiến trúc.
- Kiến trúc là câu trả lời cho: hệ thống được chia thành những phần nào, mỗi phần chịu trách nhiệm gì, các phần trao đổi với nhau bằng cách nào, và vì sao chia như vậy chứ không phải cách khác.
- Đặc điểm phân biệt quyết định kiến trúc với quyết định thông thường là CHI PHÍ ĐẢO NGƯỢC. Đổi tên một biến mất 10 giây. Đổi cách chia lớp khi đã viết 3.000 dòng mã mất nhiều ngày. Đổi mô hình dữ liệu khi đã có dữ liệu thật mất nhiều hơn nữa.
- Vì vậy tiêu chí đánh giá của học phần không phải "kiến trúc của em có hiện đại không", mà là "em có LẬP LUẬN được vì sao chọn nó, dựa trên yêu cầu cụ thể của bài toán không".

Cùng một hệ thống, hai cách mô tả

X Mô tả bằng danh sách công nghệ

“Hệ thống của em dùng React cho frontend, Node.js cho backend, PostgreSQL làm cơ sở dữ liệu và triển khai lên Render.”

- Không biết hệ thống gồm những phần nào, mỗi phần làm gì.
- Không biết quy tắc “thiết bị phải còn hạn bảo hành” được kiểm ở đâu.
- Không biết các phần trao đổi với nhau thế nào.
- Đổi React sang Vue thì mô tả thay đổi, nhưng kiến trúc thực ra không đổi.

✓ Mô tả bằng quyết định kiến trúc

- 1 **Chia phần:** 4 lớp: trình bày – nghiệp vụ – truy cập dữ liệu – lưu trữ.
- 2 **Trách nhiệm:** Mọi quy tắc nghiệp vụ (QT-01...QT-05) chỉ nằm ở lớp nghiệp vụ.
- 3 **Trao đổi:** Trình bày ↔ nghiệp vụ qua REST/JSON; nghiệp vụ gọi dữ liệu qua giao diện Repository.
- 4 **Vì sao:** NFR3: đổi một quy tắc không phải sửa quá 1 module → gom quy tắc về một lớp.

Khó đảo ngược (là kiến trúc): chọn CSDL quan hệ hay NoSQL · cách chia lớp · định dạng API giữa các phần · lược đồ dữ liệu cốt lõi.

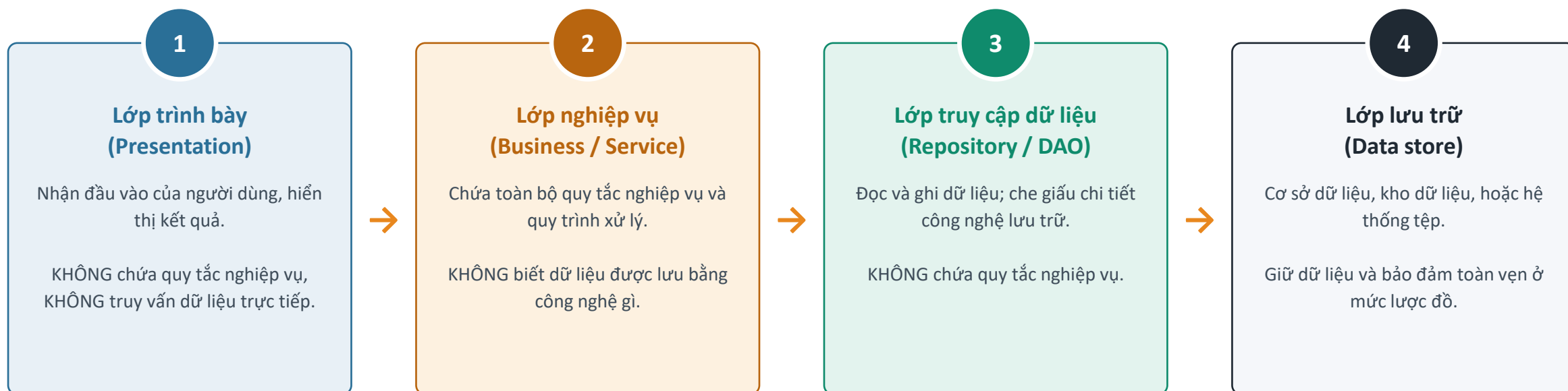
Dễ đảo ngược (là chi tiết): màu nút, tên biến, thư viện vẽ biểu đồ, phiên bản Node.js. Đừng tốn trang tài liệu BT1 cho nhóm này.

01

Nguyên lý kiến trúc và tách trách nhiệm

Kiến trúc phân lớp · ba mẫu thường dùng · phụ thuộc một chiều

Bốn lớp và trách nhiệm của từng lớp



NGUYÊN TẮC PHỤ THUỘC MỘT CHIỀU: lớp trên gọi lớp dưới, lớp dưới KHÔNG BAO GIỜ gọi lớp trên. Nếu lớp truy cập dữ liệu cần gọi lớp nghiệp vụ, đó là dấu hiệu trách nhiệm đã bị đặt sai chỗ. Vi phạm nguyên tắc này là lỗi kiến trúc bị trừ điểm ở BT1 và gây rắc rối lớn ở BT2.

Lần theo yêu cầu “Tạo phiếu bảo hành” từ màn hình xuống CSDL



Lỗi phân lớp hay gặp nhất trong mã nguồn sinh viên

✗ SAI – Controller chứa SQL và quy tắc nghiệp vụ

```
// TicketController.js (lớp trình bày)
router.post('/tickets', async (req, res) => {
  const d = await db.query(
    'SELECT * FROM device WHERE serial_no=$1',
    [req.body.serial]);
  if (monthsSince(d.purchase_date)
    > d.warranty_months) // QT-01
    return res.status(422).send('Hết hạn');
  await db.query('INSERT INTO ticket ...');
  res.status(201).end();
});
```

✓ ĐÚNG – mỗi lớp một trách nhiệm

```
// TicketController.js (trình bày)
router.post('/tickets', async (req, res, next) => {
  try {
    const t = await ticketService.create(req.body);
    res.status(201).json({ code: t.ticketCode });
  } catch (e) { next(e); } // lỗi -> HTTP 422
});

// TicketService.js (nghiệp vụ)
async create(dto) {
  const d = await deviceRepo.findBySerial(dto.serial);
  if (!d.isUnderWarranty(new Date())) // QT-01
    throw new WarrantyExpired(d.serialNo);
  return ticketRepo.save(Ticket.open(dto, d));
}
```

Lỗi thứ hai – lớp dưới gọi ngược lớp trên: TicketRepository.save() tự gọi NotificationService để gửi email. **Sửa:** Service gọi ticketRepo.save() rồi mới gọi notificationService.send(); Repository chỉ đọc và ghi dữ liệu.

Ba mẫu: giải quyết vấn đề gì, đánh đổi gì

Mẫu	Ý tưởng cốt lõi	Giải quyết vấn đề gì	Đánh đổi / khi nào KHÔNG dùng
Layered (phân lớp)	Chia hệ thống theo mức trừu tượng: trình bày – nghiệp vụ – dữ liệu. Phụ thuộc một chiều từ trên xuống.	Dễ hiểu, dễ chia việc, dễ thay một lớp mà không ảnh hưởng lớp khác. Là mặc định hợp lý cho prototype.	Với hệ thống rất lớn, gọi xuyên nhiều lớp gây rườm rà. Không phù hợp khi cần độ trễ cực thấp.
MVC (Model–View–Controller)	Tách dữ liệu và quy tắc (Model), phần hiển thị (View), phần điều phối yêu cầu (Controller).	Tách rõ phần hiển thị khỏi phần xử lý; nhiều View dùng chung một Model. Phù hợp ứng dụng có giao diện.	Controller dễ phình to thành nơi chứa mọi thứ. Không giải quyết vấn đề truy cập dữ liệu — vẫn cần thêm lớp Repository.
Repository	Đặt một lớp trung gian giữa nghiệp vụ và lưu trữ; nghiệp vụ chỉ làm việc với giao diện trừu tượng.	Đổi công nghệ lưu trữ mà không sửa lớp nghiệp vụ; dễ viết unit test bằng cách thay bằng repository giả.	Thêm một lớp trừu tượng nữa — với prototype rất nhỏ có thể là dư thừa. Cần cân nhắc, không dùng vì "nghe hay".

Ba mẫu này KHÔNG loại trừ nhau: một prototype web điển hình dùng Layered làm khung tổng thể, MVC cho lớp trình bày, và Repository cho lớp truy cập dữ liệu. Điều BT1 chấm không phải em chọn mẫu nào, mà là em GIẢI THÍCH ĐƯỢC vì sao chọn — gắn với yêu cầu phi chức năng cụ thể của bài toán em làm.

Cấu trúc thư mục là bằng chứng rõ nhất của kiến trúc

warranty-app/ · luồng L2, track SE

```
src/
├── presentation/           ← LAYERED: lớp trình bày
│   ├── controllers/       ← MVC: Controller
│   │   └── TicketController.js
│   └── views/             ← MVC: View
│       ├── ticket-list.ejs
│       └── ticket-form.ejs
├── services/              ← LAYERED: lớp nghiệp vụ
│   ├── TicketService.js
│   └── AssignService.js
├── domain/                ← MVC: Model (thực thể + quy tắc)
│   └── Ticket.js
├── repositories/          ← REPOSITORY: truy cập dữ liệu
│   ├── TicketRepository.js    (giao diện)
│   ├── PgTicketRepository.js  (cài đặt PostgreSQL)
│   └── MemTicketRepository.js  (dùng khi kiểm thử)
db/schema.sql              ← lớp lưu trữ
docs/ srs.md · architecture.drawio · erd.drawio
```

Layered

Khung tổng thể: mỗi thư mục cấp 1 trong src/ là một lớp; phụ thuộc đi từ trên xuống.

MVC

Chỉ tổ chức bên trong lớp trình bày: Controller nhận yêu cầu, View hiển thị, Model là thực thể miền.

Repository

Service chỉ biết giao diện TicketRepository. Đổi PostgreSQL sang bản trong bộ nhớ để test mà không sửa Service.

Sơ đồ thành phần: đủ chi tiết để hiện thực, đủ gọn để đọc

HỆ THỐNG: Tiếp nhận và phân loại yêu cầu bảo hành (track SE)

```
+-----+
| LỚP TRÌNH BÀY                                     |
| [ Trang tiếp nhận ] [ Danh sách phiếu ] [ Trang phân công ] |
+-----+
| v HTTP + JSON (hợp đồng API ở docs/api.md)      |
+-----+
| LỚP NGHIỆP VỤ (Service)                           |
| TicketService - tạo phiếu, kiểm quy tắc bắt buộc |
| ClassifyService - đề xuất nhóm sự cố và mức ưu tiên |
| AssignService - gán kỹ thuật viên theo tay nghề |
| >>> TOÀN BỘ quy tắc nghiệp vụ nằm trong lớp này <<< |
+-----+
| v gọi qua giao diện Repository                    |
+-----+
| LỚP TRUY CẬP DỮ LIỆU                             |
| TicketRepository CustomerRepository TechnicianRepository |
+-----+
| v SQL                                              |
+-----+
| LỚP LƯU TRỮ - PostgreSQL                         |
| ticket, ticket_status_log, customer, device, technician |
+-----+
```

CHÚ THÍCH: [] = màn hình | v = hướng phụ thuộc (một chiều)
NGOÀI PHẠM VI (WON'T): gửi SMS cho khách, đồng bộ hệ thống kế toán

Ghi nhớ

- Sơ đồ này đủ cho BT1: thấy được các phần, trách nhiệm từng phần, và cách trao đổi.
- Dòng CHÚ THÍCH là bắt buộc — mục kiểm chứng BT1 yêu cầu sơ đồ đọc được mà không cần giải thích thêm.
- Dòng NGOÀI PHẠM VI thể hiện em đã cân nhắc có ý thức, nối lại mức WON'T của MoSCoW.
- Tên bảng ở lớp lưu trữ phải khớp đúng với ERD ở thành phần 4 của BT1.
- Track DA và AI vẽ theo cùng nguyên tắc, chỉ đổi nội dung từng lớp — xem slide phần chuyên ngành.
- Vẽ bằng draw.io, lưu file .drawio gốc vào docs/.

02

Tiêu chí thiết kế tốt và cách lập luận lựa chọn

Năm tiêu chí đánh giá · cách viết ba câu giải thích gắn với yêu cầu phi chức năng

Năm câu hỏi để tự đánh giá thiết kế của em

1

RÕ RÀNG

Người đọc hiểu đúng ý em mà không cần hỏi thêm?

Mọi ký hiệu có chú thích, mọi thành phần có tên nói lên trách nhiệm.

2

NHẤT QUÁN

Cùng một khái niệm chỉ có một tên trong mọi tài liệu và mọi sơ đồ?

SRS, sơ đồ kiến trúc, ERD, wireframe dùng đúng một bộ thuật ngữ.

3

TÁCH TRÁCH NHIỆM

Mỗi thành phần làm một việc và biết càng ít về thành phần khác càng tốt?

Đổi một phần không buộc phải sửa các phần còn lại.

4

CÓ LẬP LUẬN

Mỗi quyết định nối được với một yêu cầu cụ thể?

Không có thành phần nào tồn tại chỉ vì "thường người ta làm vậy".

5

MỞ RỘNG ĐƯỢC

Thêm một chức năng tương tự thì sửa ở bao nhiêu chỗ?

Sửa một chỗ là tốt; phải sửa năm chỗ là dấu hiệu thiết kế chưa ổn.

Năm dấu hiệu trên bản nháp và cách sửa

Tiêu chí	Dấu hiệu trên sơ đồ nháp của sinh viên	Sửa thành
1 • Rõ ràng	Mũi tên không có nhãn; các ô ghi “Module 1”, “Module 2”, “Xử lý”.	Ghi giao thức trên mũi tên (HTTP/JSON, SQL); đặt tên theo trách nhiệm: TicketService, AssignService.
2 • Nhất quán	Sơ đồ ghi “Đơn sửa chữa”, ERD ghi ticket, SRS ghi “phiếu bảo hành”.	Chốt một thuật ngữ “phiếu bảo hành” (ticket) trong bảng thuật ngữ mục 1 SRS; sửa mọi sơ đồ theo.
3 • Tách trách nhiệm	Một ô “Backend” làm mọi việc: kiểm quy tắc, gửi email, truy vấn CSDL.	Tách thành lớp nghiệp vụ (3 service) và lớp truy cập dữ liệu (repository) như slide kiến trúc phân lớp.
4 • Có lập luận	Có ô “Redis cache” và “Message queue” nhưng không NFR nào cần đến.	Bỏ đi; nếu giữ thì phải có câu “Vì NFRx yêu cầu..., tôi chọn..., đánh đổi là...”.
5 • Mở rộng được	Thêm loại yêu cầu “đổi trả” phải sửa controller, service và tạo thêm 3 bảng mới.	Thêm cột request_type và bảng issue_category; loại yêu cầu mới chỉ là thêm một dòng dữ liệu.

Mẹo tự kiểm: đưa sơ đồ cho một bạn đọc 2 phút rồi nhờ bạn kể lại đường đi của yêu cầu “Tạo phiếu”. Bạn kể sai ở đâu, sơ đồ chưa rõ ở đó.

Cách viết ba câu lập luận lựa chọn kiến trúc

- **KHUÔN:** "Vì yêu cầu <mã NFR> đòi hỏi <ngưỡng cụ thể>, tôi chọn <quyết định kiến trúc>, đánh đổi là <chi phí>."
- **Ví dụ 1 — gắn với hiệu năng:**
 - Vì NFR1 yêu cầu danh sách phiếu hiển thị dưới 2 giây với 10.000 bản ghi, tôi chọn phân trang ở tầng cơ sở dữ liệu và đánh index trên (technician_id, due_date) thay vì tải toàn bộ rồi lọc ở lớp trình bày. Đánh đổi: thêm chi phí ghi khi cập nhật phiếu, chấp nhận được vì tần suất đọc cao hơn tần suất ghi.
- **Ví dụ 2 — gắn với bảo trì được:**
 - Vì NFR5 yêu cầu thêm nhóm sự cố mới mà không sửa mã nguồn, tôi đặt bảng danh mục nhóm sự cố trong cơ sở dữ liệu thay vì viết cứng trong mã. Đánh đổi: cần thêm một màn hình quản trị danh mục, xếp ở mức SHOULD của MoSCoW.
- **Ví dụ 3 — gắn với kiểm thử được:**
 - Vì BT2 yêu cầu ít nhất 3 unit test cho lớp nghiệp vụ, tôi tách lớp Repository để có thể thay bằng repository giả khi test, không cần cơ sở dữ liệu thật. Đánh đổi: thêm một lớp trừu tượng, chấp nhận được ở quy mô 5 bảng.

Ba lỗi khiến lập luận không được tính điểm

- Chỉ nêu ưu điểm chung chung: "kiến trúc phân lớp giúp code dễ bảo trì hơn" — không gắn với yêu cầu nào của bài toán em làm.
- Nêu lý do theo mốt: "em dùng microservices vì đây là kiến trúc hiện đại" — với prototype một luồng nghiệp vụ, đây là lựa chọn sai và không có lập luận.
- Không nêu đánh đổi: mọi quyết định kiến trúc đều có giá. Không nói được giá nghĩa là chưa thực sự cân nhắc.

NFR2: tiếp nhận phiếu ≤ 1 giây, kể cả khi có phân loại tự động

Phương án A – Phân loại đồng bộ



một request, chờ tuần tự

- + Đơn giản, ít thành phần; nhân viên thấy nhóm sự cố ngay.
- Nếu bộ phân loại chậm 2 giây, cả yêu cầu chậm theo → vi phạm NFR2.

Phương án B – Phân loại nền qua hàng đợi



request trả về ngay; bước 3–4 chạy nền

- + Tạo phiếu luôn trả về ngay, chắc chắn đạt NFR2.
- Thêm hàng đợi và worker cần vận hành; phiếu ở trạng thái “Chưa phân loại” vài giây; khó gỡ lỗi.

Quyết định cho prototype: Phương án A, thêm giới hạn thời gian 500 ms; quá hạn thì gán nhóm “Chưa phân loại” để điều phối viên xử lý tay.

Câu lập luận nộp BT1: “Vì NFR2 yêu cầu tiếp nhận phiếu ≤ 1 giây, tôi chọn phân loại đồng bộ có timeout 500 ms thay vì hàng đợi, đánh đổi là một tỷ lệ nhỏ phiếu phải phân loại thủ công khi bộ phân loại phản hồi chậm.”

03

Nguyên tắc mô hình hóa dữ liệu

Thực thể – thuộc tính – quan hệ · khóa chính và khóa ngoại · chuẩn hóa · bốn lỗi phổ biến

Bốn bước đi từ đặc tả yêu cầu tới mô hình dữ liệu

1

Tìm THỰC THỂ: gạch chân danh từ trong bản SRS

Đọc lại SRS và gạch chân mọi danh từ nghiệp vụ: phiếu bảo hành, khách hàng, thiết bị, kỹ thuật viên, nhóm sự cố. Mỗi danh từ có vòng đời riêng và cần lưu lại thì là một thực thể.

2

Tìm THUỘC TÍNH: hỏi "cần biết gì về thực thể này?"

Với mỗi thực thể, lấy thuộc tính từ mục 5 của SRS (ràng buộc và quy tắc nghiệp vụ) và từ wireframe. Ghi rõ kiểu dữ liệu và thuộc tính nào bắt buộc — đây là nguồn của validation ở BT2.

3

Tìm QUAN HỆ: đọc động từ nối hai danh từ

"Khách hàng TẠO phiếu bảo hành" → quan hệ một–nhiều. "Phiếu bảo hành DỪNG các linh kiện" → quan hệ nhiều–nhiều, cần bảng trung gian. Xác định lực lượng (1:1, 1:N, N:M) cho từng quan hệ.

4

Kiểm TRUY VẾT ngược: mỗi bảng phục vụ yêu cầu nào?

Đọc ngược từ mô hình dữ liệu về bảng truy vết trong SRS. Bảng nào không phục vụ yêu cầu chức năng nào thì bỏ đi. Yêu cầu nào không có bảng nào đỡ thì mô hình còn thiếu.

Gạch chân danh từ, khoanh động từ

Trích SRS – mục 3 (luồng L2)

FR1. Nhân viên tiếp nhận **TẠO** **phiếu bảo hành** cho một **khách hàng** khi khách mang **thiết bị** đến trung tâm. Mỗi phiếu ghi **số serial**, **mô tả sự cố** và **mức ưu tiên**.

FR3. Điều phối viên **GÁN** phiếu cho một **kỹ thuật viên** phù hợp với **nhóm sự cố**.

FR4. Phiếu **DỪNG** một hoặc nhiều **linh kiện**, mỗi linh kiện ghi số lượng.

FR5. Hệ thống **LƯU** **lịch sử đổi trạng thái** của phiếu: ai đổi, lúc nào.

Chú thích: **danh từ** → thực thể/thuộc tính · **ĐỘNG TỪ** → quan hệ

Danh từ trong SRS	Kết quả mô hình hóa
phiếu bảo hành	Thực thể ticket
khách hàng · thiết bị · kỹ thuật viên	customer · device · technician
số serial · mô tả sự cố · mức ưu tiên	Thuộc tính (serial_no, issue_desc, priority)
lịch sử đổi trạng thái	Thực thể ticket_status_log
nhân viên tiếp nhận · điều phối viên	Actor → một bảng app_user, cột role

Động từ → quan hệ	Lực lượng	Cách cài đặt
khách hàng TẠO phiếu	1 : N	FK customer_id ở ticket
điều phối viên GÁN cho kỹ thuật viên	1 : N	FK technician_id, cho phép NULL
phiếu DỪNG linh kiện	N : M	Bảng trung gian ticket_part
hệ thống LƯU lịch sử	1 : N	FK ticket_id ở ticket_status_log

Ba dạng chuẩn đầu tiên và ý nghĩa thực dụng

Dạng chuẩn	Quy tắc	Vi phạm trông như thế nào	Sửa thế nào
1NF	Mỗi ô chứa một giá trị đơn; không có nhóm lặp.	Cột linh_kien lưu "pin, màn hình, cáp" trong một ô text. Không truy vấn được "phiếu nào dùng pin".	Tách thành bảng ticket_part (ticket_id, part_id, so_luong).
2NF	Đã ở 1NF; mọi thuộc tính không khóa phụ thuộc vào TOÀN BỘ khóa chính.	Bảng ticket_part có khóa chính (ticket_id, part_id) nhưng lại chứa cột ten_khach_hang — chỉ phụ thuộc ticket_id.	Chuyển ten_khach_hang về bảng ticket (hoặc customer), bỏ khỏi bảng trung gian.
3NF	Đã ở 2NF; không có thuộc tính không khóa phụ thuộc vào thuộc tính không khóa khác.	Bảng ticket chứa cả technician_id và ten_technician, sdt_technician — hai cột sau phụ thuộc technician_id.	Giữ technician_id làm khóa ngoại, bỏ ten và sdt; lấy bằng phép nối khi cần.

BT1 track SE yêu cầu ERD chuẩn 3NF. PHI CHUẨN HÓA CÓ Ý THỨC vẫn được chấp nhận và thậm chí được khuyến khích ở track DA (kho dữ liệu dạng star schema chủ ý phi chuẩn hóa để đọc nhanh) — nhưng phải GHI RÕ lý do trong tài liệu. Phi chuẩn hóa mà không giải thích thì bị tính là lỗi thiết kế.

Từ một trang tính Excel đến bốn bảng 3NF

Chưa chuẩn hóa

```
phieu
-----
ma_phieu  BH-001
ten_kh    An
sdt_kh    0909..
linh_kien pin, cáp
ten_ktv    Bình
```

Một ô chứa nhiều linh kiện: không truy vấn được “phiếu nào dùng pin”.

1NF

```
ticket
  ticket_id, ten_kh,
  sdt_kh, ten_ktv

ticket_part
  ticket_id, part_id,
  ten_linh_kien,
  so_luong
```

Mỗi ô một giá trị. Nhưng ten_linh_kien chỉ phụ thuộc part_id.

2NF

```
ticket
  ticket_id, ten_kh,
  sdt_kh, ten_ktv

ticket_part
  ticket_id, part_id,
  so_luong

part
  part_id, part_name
```

Bỏ phụ thuộc bộ phận. Nhưng ten_kh phụ thuộc sdt_kh, không phụ thuộc trực tiếp ticket_id.

3NF

```
customer
  customer_id, phone,
  full_name

ticket
  ticket_id,
  customer_id (FK)

ticket_part
  ticket_id, part_id
part (giữ nguyên)
```

Bỏ phụ thuộc bắc cầu. Đổi tên khách chỉ sửa một dòng.

Kết quả: customer · ticket · part · ticket_part — mỗi sự thật được lưu đúng một lần. **Câu hỏi kiểm tra 3NF nhanh:** với mỗi cột không khóa, “nếu biết khóa chính thì có biết ngay giá trị cột này không, mà không cần đi qua cột khác?”

Năm lỗi hay gặp trong ERD của sinh viên

1

BẢNG CÔ LẬP — bảng không có quan hệ nào với phần còn lại

Mục kiểm chứng BT1 ghi rõ "không có bảng cô lập". Bảng cô lập thường là dấu hiệu em thêm bảng theo cảm tính, không xuất phát từ yêu cầu. Cách sửa: hoặc nối nó vào mô hình, hoặc bỏ nó đi.

2

THIẾU BẢNG LỊCH SỬ — ghi đè trạng thái thay vì lưu vết

Bảng phiếu chỉ có một cột trang_thai bị cập nhật liên tục, nên không trả lời được "phiếu này ở trạng thái Đang xử lý bao lâu". Cách sửa: thêm bảng ticket_status_log (ticket_id, trang_thai, thoi_diem, nguoi_thuc_hien).

3

LƯU GIÁ TRỊ TÍNH ĐƯỢC MÀ KHÔNG CÓ LÝ DO

Lưu cả ngay_tao, ngay_dong và so_ngay_xu_ly — cột thứ ba tính được từ hai cột đầu và sẽ sai khi hai cột kia thay đổi. Chỉ lưu giá trị tính được khi có lý do hiệu năng rõ ràng, và phải ghi lý do.

4

KHÔNG CÓ INDEX Ở CÁC CỘT ĐƯỢC TRUY VẤN NHIỀU

Yêu cầu phi chức năng đòi hỏi hiển thị dưới 2 giây với 10.000 bản ghi, nhưng ERD không có index nào trên cột dùng để lọc và sắp xếp. Đây là chỗ mất liên kết giữa yêu cầu phi chức năng và thiết kế.

5

DÙNG KHÓA NGHIỆP VỤ LÀM KHÓA CHÍNH MÀ KHÔNG CÂN NHẮC

Lấy số điện thoại khách hàng làm khóa chính bảng customer. Khi khách đổi số điện thoại thì phải cập nhật mọi bảng tham chiếu. Cách sửa: dùng khóa nhân tạo (id tự tăng hoặc UUID), đặt ràng buộc UNIQUE trên số điện thoại.

Một ERD nộp nháp được sửa lại như thế nào

✗ TRƯỚC – bản nháp của sinh viên

customer	ticket
-----	-----
PK phone [5]	PK ticket_id
full_name	FK phone
email	linh_kien TEXT
	trang_thai [2]
report_config [1]	ngay_tao
-----	ngay_dong
PK id	so_ngay_xu_ly [3]
json_data	(không index) [4]

✓ SAU – đạt mục kiểm chứng track SE

customer	ticket
-----	-----
PK customer_id	PK ticket_id
phone UNIQUE	FK customer_id
full_name	status, due_date
	received_at, closed_at
ticket_status_log	INDEX (status, due_date)
-----	-----
PK log_id	ticket_part part
FK ticket_id	PK,FK ticket_id PK part_id
to_status	PK,FK part_id
changed_at	quantity

- [1] Bảng cô lập report_config không nối với bảng nào, không phục vụ FR nào → bỏ.
 - [2] Ghi đề trang_thai → thêm ticket_status_log để biết phiếu ở mỗi trạng thái bao lâu.
 - [3] so_ngay_xu_ly tính được từ received_at, closed_at → bỏ cột.
 - [4] Thêm INDEX (status, due_date) phục vụ NFR1 (< 2 giây với 10.000 phiếu).
 - [5] Khóa chính là số điện thoại → khóa nhân tạo customer_id, phone chỉ UNIQUE.
- Thêm:** linh_kien TEXT (vi phạm 1NF) được tách thành ticket_part + part.

04

Wireframe và tài liệu thiết kế

Vai trò wireframe · ba màn hình BT1 yêu cầu · cấu trúc SDD và mức rút gọn của học phần

Wireframe: cần gì và không cần gì

- **MỤC ĐÍCH** của wireframe ở học phần này: chứng minh em đã nghĩ tới việc người dùng **THẤY GÌ** và **LÀM GÌ**, và những thứ đó nhất quán với đặc tả.
- **CẦN có:**
 - Đủ các trường dữ liệu mà yêu cầu chức năng đòi hỏi — đối chiếu được với ERD.
 - Các hành động chính của người dùng (nút, liên kết) tương ứng với các use case.
 - Chỗ hiển thị thông báo lỗi cho các luồng ngoại lệ đã đặc tả.
 - Nhãn dùng đúng thuật ngữ trong bảng thuật ngữ của SRS.
- **KHÔNG cần:**
 - Màu sắc, phông chữ, hình ảnh, logo — đây không phải môn thiết kế đồ họa.
 - Giao diện chạy được hay prototype tương tác. Bản vẽ tĩnh là đủ.
 - Vẽ bằng công cụ chuyên nghiệp. Figma, Balsamiq, draw.io, hoặc **VẼ TAY RỒI CHỤP ẢNH** đều được chấp nhận.

Ba màn hình BT1 yêu cầu

- [SE] Ba màn hình chính của luồng: thường là màn hình danh sách, màn hình tạo/sửa, và màn hình chi tiết.
- [DA] Ba khung dashboard: bố cục các biểu đồ, bộ lọc, và chỉ số tổng hợp — kèm ghi chú nguồn dữ liệu của từng khung.
- [AI] Ba bước pipeline được hình dung: giao diện nạp dữ liệu, màn hình xem kết quả suy luận, và bảng chỉ số đánh giá mô hình.
- Yêu cầu chung: mỗi màn hình phải nối được về ít nhất một use case, và mọi trường dữ liệu phải có trong mô hình dữ liệu.

Mỗi trường trên màn hình phải chỉ ra được cột dữ liệu và yêu cầu

Tạo phiếu bảo hành

1

Số điện thoại khách

0909 123 456

Tra cứu

2

Họ tên khách

Nguyễn Văn An (tự điền)

3

Số serial thiết bị

SN-A52-778901

4

Mô tả sự cố

Máy không lên nguồn sau khi sạc...

5

Mức ưu tiên

Cao

Trung bình

Thấp

6

⚠ Thiết bị SN-A52-778901 đã hết hạn bảo hành (QT-01).

Hủy

Tạo phiếu

#	Trên màn hình	Cột trong ERD	Nguồn
1	Số điện thoại khách	customer.phone (UNIQUE)	FR1 · US1
2	Họ tên khách	customer.full_name	FR1
3	Số serial thiết bị	device.serial_no	FR1 · QT-01
4	Mô tả sự cố	ticket.issue_desc (≥ 10 ký tự)	FR1 · QT-03
5	Mức ưu tiên	ticket.priority	FR2
6	Thông báo lỗi	— (không lưu)	UC01 · luồng E1
	Nút “Tạo phiếu”	INSERT ticket + ticket_status_log	UC01 · US1

Kiểm hai chiều: mọi trường trên màn hình có cột tương ứng; ngược lại mọi cột NOT NULL của ticket đều được nhập hoặc được hệ thống tự sinh (ticket_code, due_date, status).

SDD đầy đủ và phần áp dụng ở Chuyên đề tốt nghiệp 1

Mục của một bản SDD đầy đủ	CĐTN1 (học phần này)	Ghi chú
Tổng quan kiến trúc và các quyết định chính	CÓ — bắt buộc	Sơ đồ thành phần/phân lớp + ≥ 3 câu lập luận gắn với NFR. Thành phần 3 của BT1.
Mô hình dữ liệu (logic và vật lý)	CÓ — bắt buộc	ERD 4–6 bảng + DDL skeleton [SE] / lược đồ DWH [DA] / dataset schema [AI]. Thành phần 4 của BT1.
Thiết kế giao diện người dùng	CÓ — mức rút gọn	Wireframe 3 màn hình chính. Thành phần 5 của BT1.
Thiết kế chi tiết từng module (lớp, hàm, thuật toán)	KHÔNG bắt buộc	Ở quy mô prototype, mã nguồn tự thân là tài liệu chi tiết. Chỉ mô tả thuật toán nếu nó không hiển nhiên.
Thiết kế tích hợp với hệ thống ngoài	KHÔNG	Case study không có hệ thống ngoài trong phạm vi CĐTN1. Thuộc CĐTN2.
Thiết kế triển khai, hạ tầng, monitoring	KHÔNG — chỉ đóng gói	BT3 chỉ yêu cầu đóng gói tái lập được và Deployment Guide. Hạ tầng và vận hành thuộc CĐTN2.
Thiết kế bảo mật đầy đủ, mô hình hóa mối đe dọa	KHÔNG — chỉ checklist	BT3 yêu cầu checklist bảo mật cơ bản đã tự đánh giá. Mô hình hóa mối đe dọa thuộc CĐTN2.

Nhắc lại nguyên tắc phạm vi: làm thừa KHÔNG được cộng điểm, mà lấy đi thời gian của phần được chấm. Ba mục đầu là ba thành phần của BT1; bốn mục sau không nộp ở học phần này.

05

Phần chuyên ngành và chốt yêu cầu Bài tập 1

ERD chuẩn 3NF [SE] · lược đồ star/snowflake và SCD [DA] · ML pipeline [AI] · năm thành phần BT1

Mô hình dữ liệu theo chuyên ngành: thành phần 4 của BT1

Cùng nguyên tắc chung (thực thể – thuộc tính – quan hệ – truy vết ngược), khác ký pháp và khác mục tiêu tối ưu. Tài liệu tự học buổi 5 theo track có ví dụ mẫu hoàn chỉnh trên luồng L2.

SE ERD quan hệ chuẩn 3NF

- Yêu cầu BT1: ERD 4–6 bảng, mỗi bảng có khóa chính, ≥ 2 quan hệ khóa ngoại, không có bảng con lập
- Chuẩn 3NF; phi chuẩn hóa phải ghi rõ lý do
- Ghi rõ index trên các cột dùng để lọc và sắp xếp, gắn với NFR hiệu năng
- Kèm SQL DDL skeleton: CREATE TABLE với kiểu dữ liệu, NOT NULL, FOREIGN KEY, UNIQUE
- Mục tiêu tối ưu: toàn vẹn dữ liệu và tránh dư thừa

DA Lược đồ kho dữ liệu

- Yêu cầu BT1: ≥ 1 bảng fact + 2–3 bảng dimension; mỗi cột ghi rõ kiểu dữ liệu và mô tả
- Chọn star schema (đơn giản, đọc nhanh) hoặc snowflake (chuẩn hóa hơn) — và giải thích lựa chọn
- Xác định GRAIN của bảng fact: một dòng đại diện cho cái gì (một phiếu? một lần đổi trạng thái?)
- Xử lý dimension thay đổi theo thời gian (SCD) — nêu loại SCD chọn dùng và lý do
- Mục tiêu tối ưu: tốc độ đọc và khả năng tổng hợp nhiều chiều — chủ ý phi chuẩn hóa

AI ML pipeline & dataset schema

- Yêu cầu BT1: sơ đồ ML pipeline + mô tả dataset nguồn, ghi rõ input/output, loại bài toán, số mẫu, metric dự kiến
- Sơ đồ pipeline: nạp dữ liệu $\rightarrow$ làm sạch $\rightarrow$ tạo đặc trưng $\rightarrow$ huấn luyện $\rightarrow$ đánh giá $\rightarrow$ phục vụ
- Dataset schema: tên đặc trưng, kiểu, cách tính, nguồn gốc; định nghĩa nhãn và cách gán
- Nêu rõ cách chia train/validation/test và lý do (đặc biệt nếu dữ liệu có yếu tố thời gian)
- Mục tiêu tối ưu: tránh rò rỉ dữ liệu (data leakage) và bảo đảm tái lập được (ghi random seed)

Một bài toán, ba mục tiêu tối ưu khác nhau

SE Bảng quan hệ 3NF

```
CREATE TABLE ticket (  
  ticket_id BIGSERIAL  
    PRIMARY KEY,  
  customer_id BIGINT  
    NOT NULL REFERENCES  
    customer,  
  status VARCHAR(20)  
    NOT NULL,  
  due_date TIMESTAMPTZ  
    NOT NULL  
);
```

Câu hỏi thiết kế chính:

Dữ liệu có toàn vẹn, không dư thừa không?

DA Bảng fact + dimension

```
fact_ticket  
  grain: 1 dòng = 1 phiếu  
  date_key, center_key  
  category_key, tech_key  
  so_gio_xu_ly (measure)  
  qua_han 0/1 (measure)  
  
dim_technician  
  SCD loại 2:  
  hieu_luc_tu/den
```

Câu hỏi thiết kế chính:

Báo cáo đọc có nhanh, đúng theo thời gian không?

AI Dataset schema

```
issue_desc text (input)  
category label (8 lớp)  
  
nhãn lấy từ category_id  
do điều phối viên gán  
  
chia theo thời gian:  
train: trước 06/2026  
test : từ 06/2026
```

Câu hỏi thiết kế chính:

Có rõ ràng dữ liệu, nhãn có đáng tin không?

Mười một mục kiểm chứng của Bài tập 1

- | | | | |
|--------------------------|---|--------------------------|---|
| ☐ | Có đủ 5 đầu mục: SRS, Use Case, kiến trúc, mô hình dữ liệu, wireframe. | ☐ | Sơ đồ kiến trúc có chú thích, đọc được độc lập; ghi rõ cách các lớp trao đổi. |
| ☐ | SRS đủ 6 mục; ≥ 4 yêu cầu chức năng có mã; ≥ 3 yêu cầu phi chức năng CÓ NGƯỜI SỐ. | ☐ | Có ≥ 3 câu giải thích lựa chọn kiến trúc, mỗi câu gắn với một yêu cầu phi chức năng và nêu đánh đổi. |
| ☐ | 5–7 User Story chuẩn INVEST, đã gán mức MoSCoW. | ☐ | Mô hình dữ liệu đạt yêu cầu track: ERD 4–6 bảng, ≥ 2 khóa ngoại, không bảng cô lập [SE] / ≥ 1 fact + 2 dimension [DA] / pipeline + dataset schema [AI]. |
| ☐ | Mỗi yêu cầu chức năng truy vết được tới ít nhất 1 User Story (bảng truy vết mục 6). | ☐ | Wireframe 3 màn hình chính, mọi trường dữ liệu đối chiếu được với mô hình dữ liệu. |
| ☐ | Use Case Diagram ≥ 1 actor, ≥ 5 use case, có ranh giới hệ thống và chú thích. | ☐ | Thuật ngữ nhất quán giữa SRS và MÔ sơ đồ; có bảng khai báo sử dụng công cụ AI. |
| ☐ | Đặc tả use case quan trọng nhất: luồng chính + ≥ 1 luồng ngoại lệ. | | |

Hình thức nộp: 1 file PDF gộp toàn bộ + link GitHub repo (chứa docs/srs.md và các file .drawio gốc). Nộp qua LMS trước khi kết thúc buổi 6. · Bốn mục tô đỏ là bốn mục sinh viên trượt nhiều nhất.

Chốt buổi 5 và chuẩn bị cho buổi 6

Tự học, tự nghiên cứu (4 giờ)

- Hoàn thiện sơ đồ kiến trúc và viết ≥ 3 câu lập luận theo khuôn "Vì NFR... tôi chọn... đánh đổi là...".
- Hoàn thiện mô hình dữ liệu theo track; soi lại năm lỗi ERD đã học.
- Vẽ wireframe 3 màn hình chính; đối chiếu từng trường với mô hình dữ liệu.
- Rà soát NHẤT QUÁN THUẬT NGỮ: lập bảng thuật ngữ ở mục 1 của SRS, rồi kiểm từng sơ đồ.
- Đọc tài liệu tự học buổi 5 theo track và ví dụ mẫu đã điền sẵn.
- Gộp toàn bộ thành 1 file PDF nháp — mang đến buổi 6 để hoàn thiện và nộp.

Buổi 6 (thực hành, 3 tiết) — Workshop thiết kế và ★ NỘP BÀI TẬP 1

- Hoàn thiện sơ đồ kiến trúc, mô hình dữ liệu và wireframe tại lớp.
- Rà soát tính nhất quán SRS $\leftrightarrow$ thiết kế theo 11 mục kiểm chứng.
- Giảng viên đối chiếu bảng tiêu chí kỹ thuật tối thiểu BT1 trực tiếp trên sản phẩm.
- ★ NỘP BÀI TẬP 1 qua LMS trước khi kết thúc buổi — 10% điểm học phần.
- Mang theo: bản PDF nháp, laptop, file .drawio gốc.